```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract Bank {
mapping(address => uint256) public balances;
// Deposit ETH into the contract
function depositMoney() public payable {
require(msg.value > 0, “Amount must be greater than zero”);
balances[msg.sender] += msg.value;
}
// Withdraw ETH from the contract
function withdrawMoney(uint256 amount) public {
require(amount > 0, “Amount must be greater than zero”);
require(balances[msg.sender] >= amount, “Insufficient balance”);
balances[msg.sender] -= amount;
(bool success, ) = payable(msg.sender).call{value: amount}(“”);
require(success,”Transfer failed”);
}
// Show the caller’s balance
function showBalance() public view returns (uint256) {
return balances[msg.sender];
}
}
explain code
```

This is a **Bank Smart Contract written in Solidity**. It allows users to:

1. Deposit ETH (Ethereum cryptocurrency) into the smart contract.
2. Withdraw their deposited ETH.
3. Check their deposited balance.

Let’s understand the code line by line in simple language, suitable for your BCT practical and viva.

1. SPDX License and Solidity Version

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
```

- `SPDX-License-Identifier: MIT`: Specifies the license under which the code is distributed.
- `pragma solidity ^0.8.0`: Specifies that the code requires Solidity version 0.8.0 or above.

2. Contract Declaration

```
contract Bank {
```

- `contract` is a keyword used to create a smart contract.

- `Bank` is the name of our smart contract.
- All variables and functions are written inside this contract.

3. Mapping (Storing User Balances)

```
mapping(address => uint256) public balances;
```

This mapping stores the balance of every user.

- `mapping`: Used to store data in key-value pairs.
- `address`: Represents the Ethereum wallet address of a user.
- `uint256`: Stores the balance as a positive integer.
- `public`: Allows users to read the mapping directly.

Example:

User address	Balance
0x123…	2 ETH
0x456…	5 ETH
0x789…	1 ETH

Each wallet address has its own balance.

4. Deposit Money Function

```
function depositMoney() public payable {
    require(msg.value > 0, "Amount must be greater than zero");
    balances[msg.sender] += msg.value;
}
```

This function allows users to deposit ETH into the smart contract.

Explanation:

- `function depositMoney()`: Creates a function named depositMoney.
- `public`: Anyone can call this function.
- `payable`: Allows the function to receive ETH.
- `msg.value`: Represents the amount of ETH sent with the transaction.
- `require()`: Checks a condition before executing the next statement.
- `msg.sender`: Represents the wallet address of the person calling the function.
- `+=`: Adds the deposited amount to the user's existing balance.

Example:

If a user deposits 2 ETH, their balance becomes 2 ETH.

If they deposit another 1 ETH, their balance becomes 3 ETH.

5. Withdraw Money Function

```
function withdrawMoney(uint256 amount) public {
    require(amount > 0, "Amount must be greater than zero");
    require(balances[msg.sender] >= amount, "Insufficient balance");

    balances[msg.sender] -= amount;

    (bool success, ) = payable(msg.sender).call{value: amount}("");
    require(success, "Transfer failed");
}
```

This function allows users to withdraw their deposited ETH.

Explanation:

1. `uint256 amount`: Accepts the amount the user wants to withdraw.
2. `require(amount > 0)`: Checks that the withdrawal amount is greater than zero.
3. `require(balances[msg.sender] >= amount)`: Checks whether the user has sufficient balance.
4. `balances[msg.sender] -= amount`: Deducts the withdrawal amount from the user's balance.
5. `payable(msg.sender)`: Converts the user's address into an address capable of receiving ETH.
6. `.call{value: amount}("")`: Transfers ETH to the user's wallet.
7. `bool success`: Stores whether the transfer was successful.
8. `require(success)`: Displays an error if the transfer fails.

Example:

Suppose your balance is 5 ETH.

- You withdraw 2 ETH.
- Your wallet receives 2 ETH.
- Your remaining contract balance becomes 3 ETH.

Important: The balance is deducted before transferring ETH. This follows the Checks-Effects-Interactions security pattern and helps prevent reentrancy attacks.

6. Show Balance Function

```
function showBalance() public view returns (uint256) {
    return balances[msg.sender];
}
```

This function displays the balance of the user who calls it.

Explanation:

- `public`: Anyone can call this function.

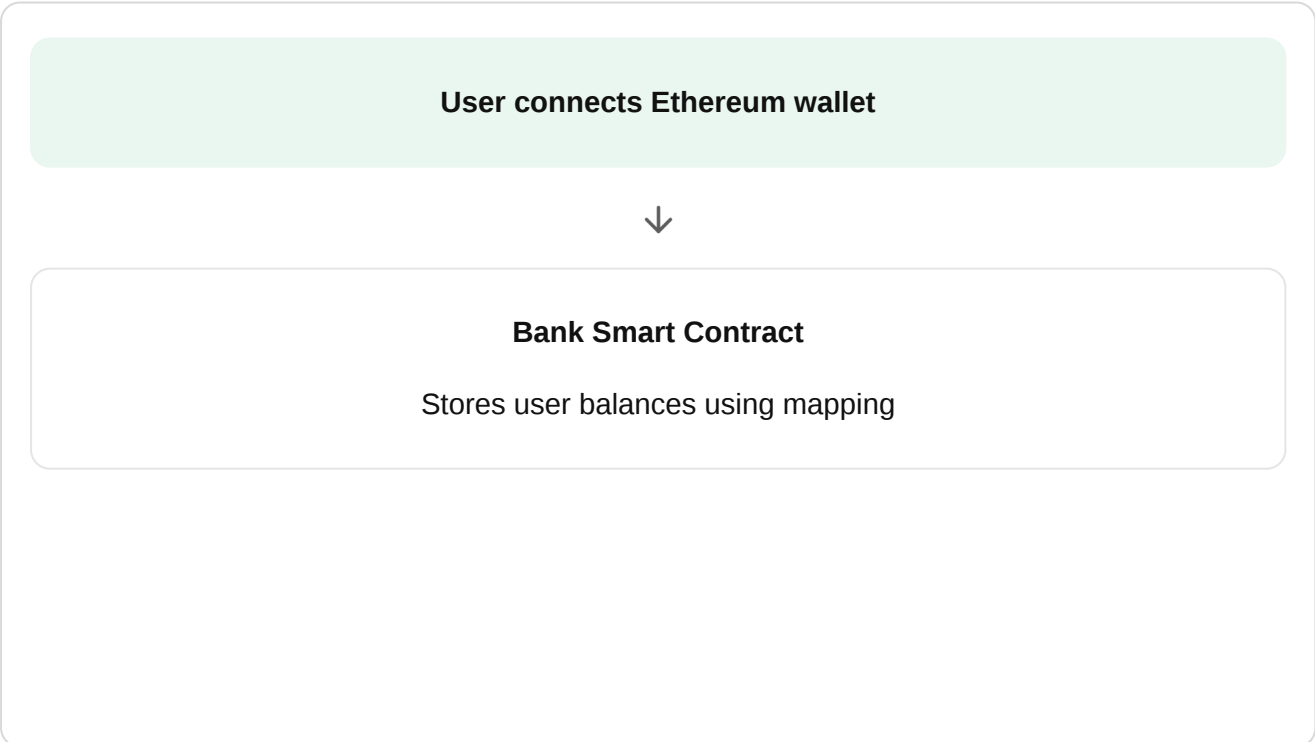
- `view`: Indicates that the function only reads data and does not modify blockchain storage.
- `returns (uint256)`: Returns a numerical value.
- `balances[msg.sender]`: Retrieves the balance of the current user's wallet.

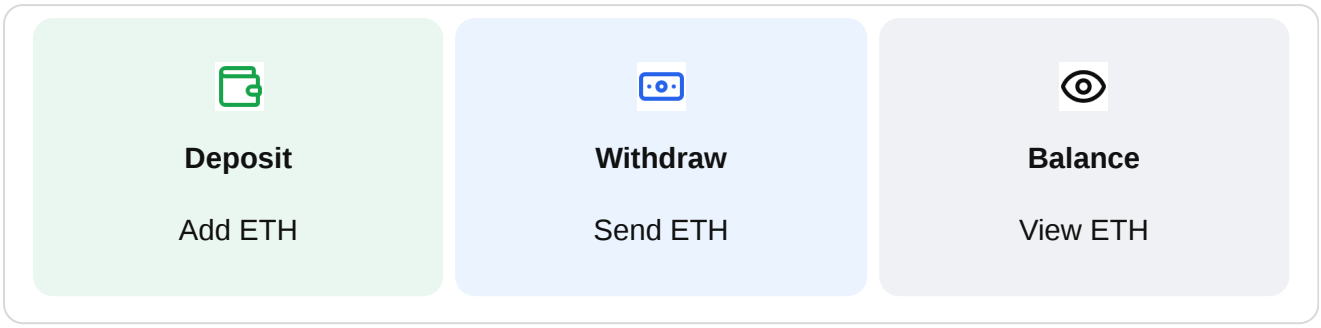
Example: If your deposited balance is 3 ETH, this function returns 3 ETH in Wei units.

7. Important Solidity Keywords for Viva

Keyword	Meaning
`contract`	Creates a smart contract
`mapping`	Stores key-value pairs
`address`	Ethereum wallet address
`uint256`	Unsigned 256-bit integer
`public`	Accessible externally
`payable`	Allows receiving ETH
`msg.sender`	Address of the caller
`msg.value`	ETH sent with transaction
`require()`	Checks a condition
`view`	Reads data without modifying state
`returns`	Specifies the returned value
`call()`	Sends ETH or makes an external call

8. How the Smart Contract Works





9. Important Viva Questions and Answers

Q1. What is Solidity?

Solidity is a high-level programming language used to write smart contracts on Ethereum.

Q2. What is a smart contract?

A smart contract is a self-executing program deployed on a blockchain.

Q3. What is the use of mapping?

Mapping stores data in key-value pairs. Here, it stores the balance of each user's wallet address.

Q4. Why do we use payable?

We use payable to allow a function to receive Ether.

Q5. What is msg.sender?

It represents the Ethereum address of the user who calls the function.

Q6. What is msg.value?

It represents the amount of Ether sent along with a payable transaction.

Q7. What is the difference between view and payable?

- `view` reads data without modifying blockchain storage.
- `payable` allows a function to receive Ether.

Q8. What happens if a user withdraws more money than their balance?

The `require()` statement stops execution and displays "Insufficient balance".

Q9. What is Wei?

Wei is the smallest unit of Ether.

1 ETH = 10^{18} Wei.

Q10. Which network can be used to deploy this contract?

We can deploy it on Ethereum Sepolia Testnet using Remix IDE and MetaMask.

---

10. Practical Demonstration Example

Suppose you deploy this contract on Sepolia Testnet.

Operation	Amount	Remaining balance
Initial balance	0 ETH	0 ETH
Deposit	2 ETH	2 ETH
Deposit	1 ETH	3 ETH
Withdraw	1 ETH	2 ETH
Show balance	—	2 ETH

Expected final balance: 2 ETH

One important point for your practical: The contract's `showBalance()` returns the user's internal deposited balance, not the wallet's total ETH balance. Also, transaction gas fees are paid separately by the wallet.

In one sentence for your examiner:

"This Solidity smart contract implements a basic banking system on Ethereum where users can deposit Ether, withdraw Ether, and check their individual balances using mapping, payable functions, and address-based transactions."

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
contract StudentData {
// Structure to store student information
struct Student {
uint256 rollNo;
string name;
uint256 marks;
}
// Array to store multiple students
Student[] public students;
// Add student details
function addStudent(
uint256 _rollNo,
string memory _name,
uint256 _marks
) public {
students.push(Student(_rollNo, _name, _marks));
}
```

```
// Return details of a student using array index
function getStudent(uint256 index)
public
view
returns (uint256, string memory, uint256)
{
require(index < students.length, “Invalid student index”);
Student memory s = students[index];
return (s.rollNo, s.name, s.marks);
}
// Return total number of students
function getStudentCount() public view returns (uint256) {
return students.length;
}
// Fallback function: handles calls that do not match a function
fallback() external payable {
}
// Receive function: handles plain Ether transfers
receive() external payable {
}
}
now this of bct
```

This is a **Student Data Management Smart Contract written in Solidity**. It uses three important Solidity concepts:

1. **Structures (structs)** – To store student information.
2. **Arrays** – To store multiple student records.
3. **Fallback and Receive functions** – To handle transactions that do not call normal functions or send Ether directly.

Let’s understand the code line by line in simple language for your BCT practical and viva.

1. SPDX License and Solidity Version

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
```

- `SPDX-License-Identifier: MIT`: Specifies the license of the code.
- `pragma solidity ^0.8.20`: Specifies that Solidity compiler version 0.8.20 or above is required.

2. Contract Declaration

```
contract StudentData {
```

- `contract` is a keyword used to create a smart contract.
- `StudentData` is the name of our smart contract.
- All variables and functions are written inside this contract.

3. Structure (Struct)

```
struct Student {
    uint256 rollNo;
    string name;
    uint256 marks;
}
```

A structure is used to group different types of data under one name.

Here, we create a structure called `Student` containing three variables:

Variable	Data type	Purpose
rollNo	uint256	Stores student roll number
name	string	Stores student name
marks	uint256	Stores student marks

Example:

A student record can contain:

- Roll Number: 101
- Name: Rahul
- Marks: 85

All three values are stored together as one Student structure.

4. Array Declaration

```
Student[] public students;
```

- `Student[]`: Creates an array of Student structures.
- `students`: Name of the array.
- `public`: Allows anyone to read individual array elements through the automatically generated getter.

Why do we use arrays?

To store multiple students in one smart contract.

Example:

Array index	Roll No	Name	Marks
0	101	Rahul	85
1	102	Amit	90
2	103	Priya	88

Remember: Array indexing starts from 0, not 1.

5. Add Student Function

```
function addStudent(
    uint256 _rollNo,
    string memory _name,
    uint256 _marks
) public {
    students.push(Student(_rollNo, _name, _marks));
}
```

This function adds a new student's information to the array.

Explanation:

- `function addStudent()`: Creates a function to add student details.
- `_rollNo`: Accepts the student's roll number.
- `_name`: Accepts the student's name.
- `memory`: Temporarily stores the string while the function executes.
- `_marks`: Accepts the student's marks.
- `public`: Allows anyone to call this function.
- `students.push()`: Adds a new student record at the end of the array.
- `Student(...)`: Creates a new Student structure using the given values.

Example:

If we call:

```
addStudent(101, "Rahul", 85);
```

The following record is added:

```
Roll No = 101
Name = Rahul
Marks = 85
```

6. Get Student Function

```
function getStudent(uint256 index)
    public
    view
```

```
    returns (uint256, string memory, uint256)
{
    require(index < students.length, "Invalid student index");

    Student memory s = students[index];
    return (s.rollNo, s.name, s.marks);
}
```

This function retrieves the details of a student using their array index.

Explanation:

- `uint256 index`: Accepts the array index.
- `public`: Anyone can call this function.
- `view`: Reads data without modifying blockchain storage.
- `returns`: Specifies the values returned by the function.
- `require()`: Checks whether the given index exists in the array.
- `students.length`: Returns the total number of students.
- `Student memory s`: Temporarily stores the selected student's information.
- `students[index]`: Accesses the student at the given index.
- `return`: Returns roll number, name and marks.

Example:

If you call:

```
getStudent(0);
```

Output:

```
101
Rahul
85
```

If you enter an invalid index, the transaction or call will fail with the error:

```
Invalid student index
```

7. Get Student Count Function

```
function getStudentCount() public view returns (uint256) {
    return students.length;
}
```

This function returns the total number of students stored in the array.

- `view`: Reads data without changing it.
- `returns (uint256)`: Returns a numerical value.

- `students.length`: Gives the total number of records.

Example:

If you have added 3 students, the output will be:

```
3
```

8. Fallback Function

```
fallback() external payable {
}
```

The fallback function is automatically executed when:

- Someone calls a function that does not exist in the contract.
- Someone sends a transaction containing unknown function-call data.

Explanation:

- `fallback()`: Defines the fallback function.
- `external`: Can be called from outside the contract.
- `payable`: Allows it to receive Ether.
- `{ }`: The function has no additional instructions.

Example: If someone calls a non-existing function, Solidity executes the fallback function instead of a matching normal function.

9. Receive Function

```
receive() external payable {
}
```

The receive function handles direct Ether transfers to the smart contract when the transaction contains no data.

Explanation:

- `receive()`: Handles plain Ether transfers.
- `external`: Can be called externally.
- `payable`: Allows the contract to receive Ether.

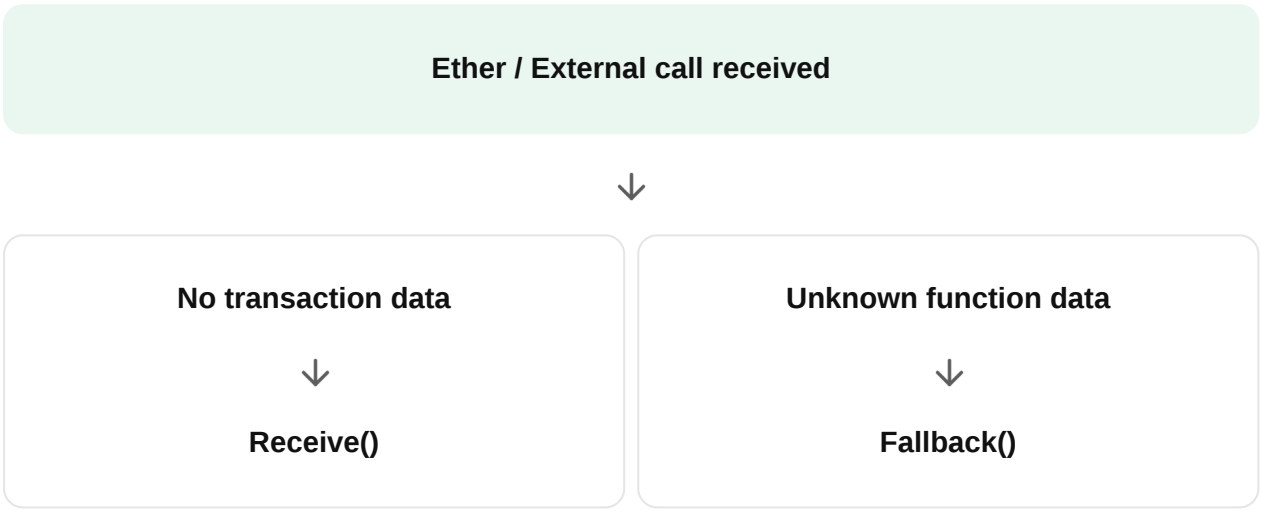
Example: If someone sends 1 ETH directly to the contract address without calling any function, the receive function executes.

10. Difference Between Fallback and Receive

This is a very important BCT viva question.

Fallback Function	Receive Function
Executes when no matching function is found	Executes on plain Ether transfer
Can handle unknown function calls	Handles Ether sent without data
Uses `fallback()`	Uses `receive()`
Can be payable	Must be payable

How they work



11. Important Solidity Keywords

Keyword	Meaning
`struct`	Groups different variables together
`uint256`	Stores unsigned integer values
`string`	Stores text
`memory`	Temporary data storage
`array`	Stores multiple elements
`push()`	Adds an element to an array
`length`	Returns the number of array elements
`public`	Allows external access
`view`	Reads data without modifying state
`returns`	Specifies returned values
`require()`	Checks a condition
`fallback()`	Handles unmatched function calls
`receive()`	Handles plain Ether transfers

Keyword	Meaning
`payable`	Allows receiving Ether

12. Practical Demonstration Using Remix IDE

After deploying your contract, try these operations:

Step	Function	Input	Expected result
1	addStudent	101, Rahul, 85	Student added
2	addStudent	102, Amit, 90	Student added
3	addStudent	103, Priya, 88	Student added
4	getStudent	0	101, Rahul, 85
5	getStudent	1	102, Amit, 90
6	getStudentCount	No input	3

Note: To test `receive()` and `fallback()`, deploy the contract and send Ether to its address using Remix's Value field. For fallback testing, send non-empty transaction data that does not match a valid function selector.

Also, this contract stores student data but does not implement access control, so anyone can add records. The fallback and receive functions accept ETH but do not record individual deposits.

---

13. Important BCT Viva Questions

Q1. What is a structure in Solidity?

A structure is a user-defined data type that groups multiple variables of different data types under one name.

Q2. Why do we use arrays?

Arrays are used to store multiple values of the same data type. Here, we store multiple Student structures.

Q3. What is the use of push()?

The push function adds a new element at the end of a dynamic array.

Q4. What is memory in Solidity?

Memory is temporary data storage that exists only during function execution.

Q5. What is the difference between storage and memory?

- Storage: Permanently stores data on the blockchain.

- Memory: Temporarily stores data during function execution.

Q6. What is fallback in Solidity?

Fallback is a special function that executes when a call does not match any existing function or when certain unmatched calldata is sent.

Q7. What is receive in Solidity?

Receive is a special function that executes when Ether is sent directly to a contract without any transaction data.

Q8. Why do we use view?

View is used when a function only reads blockchain data and does not modify it.

Q9. What is students.length?

It returns the total number of students stored in the array.

Q10. Why do we use require()?

It checks a condition and stops execution if the condition is false.

Q11. What is the difference between static and dynamic arrays?

- Static array: Has a fixed size.
- Dynamic array: Can grow or shrink in size.

Q12. Can a smart contract store Ether?

Yes. A smart contract can receive and hold Ether through payable functions such as receive and fallback.

---

14. Short Explanation to Speak in Your Practical Exam

You can memorize this paragraph:

“This program implements a Student Data Management System using Solidity. First, I created a structure named Student to store roll number, name and marks. Then I created a dynamic array to store multiple student records. The addStudent function inserts new student details into the array. The getStudent function retrieves student details using an array index, and getStudentCount returns the total number of students. Finally, I implemented fallback and receive functions to handle unknown function calls and direct Ether transfers. The contract can be deployed on the Ethereum Sepolia test network using Remix IDE and MetaMask.”

My suggestion: Understand `struct`, `array`, `push()`, `fallback()` and `receive()` particularly well. These are the five main concepts your examiner is likely to ask about in this practical.